

Garmin apps: Tennis tracker

Building a tennis tracker app for a Garmin watch like the Vivoactive 6 series involves working within Garmin's ecosystem—specifically using their SDK and programming language. Here's a clear roadmap so you know exactly what you're getting into and how to approach it.

1. Understand the Platform

Garmin apps run on their **Connect IQ** platform, not standard Android/iOS. You'll be using:

- **Garmin Connect IQ** (SDK + app ecosystem)
- **Monkey C** (their custom language)

Apps can be:

- **Data Fields** (extend existing sports)
- **Apps** (custom activity like tennis)
- **Widgets** (glanceable info)
- **Watch Faces**

👉 For a tennis tracker, you likely want an **App** (full control over tracking and UI).

2. Set Up Your Dev Environment

Install tools

- Connect IQ SDK
- Connect IQ Simulator
- Visual Studio Code + Connect IQ extension

Official docs:

- Developer portal: <https://developer.garmin.com/connect-iq/>

3. Define What Your Tennis Tracker Does

Decide your MVP (minimum viable product). Common tennis features:

Core tracking

- Match timer
- Score tracking (games/sets)
- Serve tracking
- Rally duration

Sensor-based features

- Swing detection (accelerometer)
- Movement (steps, distance)
- Heart rate

👉 Example idea:

- Detect forehand/backhand swings via wrist motion
- Count shots per rally
- Track calories and intensity

4. App Structure (High Level)

In Monkey C, your app will look something like:

```
class TennisApp extends Application.AppBase {

    function initialize() {
        AppBase.initialize();
    }

    function onStart() {
        // Initialize sensors, variables
    }

    function onUpdate(dc) {
        // Draw UI (score, time, stats)
    }

    function onStop() {
        // Save data
    }
}
```

You'll also use:

- **Sensors API** → accelerometer, heart rate
- **Activity API** → track workouts
- **Storage API** → save match data

5. Handling Tennis-Specific Logic

A. Score System

Tennis scoring is unique:

- 0 → 15 → 30 → 40 → game

- Deuce / Advantage

Build a simple state machine:

```
if playerScore == 40 and opponentScore == 40:
    state = DEUCE
```

B. Swing Detection (Hard but Cool)

Use accelerometer:

- Detect spikes in wrist acceleration
- Classify:
 - Forehand
 - Backhand
 - Serve

Garmin gives raw sensor data, but you'll need to:

- Filter noise
- Set thresholds
- Possibly use simple heuristics (not ML at first)

C. UI Constraints

Garmin watches have:

- Small screens
- Limited input (touch/buttons)

So design:

- Big buttons (score +1)
- Swipe for stats
- Minimal text

6. Test in Simulator + Real Device

Use:

- Connect IQ Simulator (fast iteration)
- Then deploy to your watch

Test:

- Responsiveness during movement
- Battery usage
- Accuracy of swing detection

7. Export & Sync Data

You can:

- Save locally on the watch
- Sync with Garmin Connect
- Send to phone via Bluetooth

Advanced option:

- Build a companion mobile app to analyze matches

8. Publish (Optional)

Publish on:

- Garmin Connect IQ Store

You'll need:

- App icon
- Description
- Device compatibility (ensure Vivoactive series support)

Practical Tips

- Start simple: score + timer
- Add swing detection later (it's the hardest part)
- Battery optimization matters a lot
- Look at open-source Connect IQ apps for examples

If You Want, I Can Help You Plan It

Tell me:

- Beginner / intermediate / experienced coder?
- Do you want swing detection or just scoring?

And I can sketch a **step-by-step build plan** or even **starter code** tailored to you.

Here's a **structured, realistic build plan** for your Garmin tennis tracker app with scoring, match formats, and health sync. I'll keep it practical so you can actually build it step by step using **Garmin Connect IQ** and **Monkey C**.



Phase 1 — Define the Core Architecture

Goal

A **Tennis Activity App** that:

- Tracks score (including *unforced errors* & *double faults*)
- Supports match formats (Bo3, tiebreaks, super tiebreak)
- Records health metrics during play

Core Modules

Split your app into **4 main components**:

1. Match Engine (logic brain)

Handles:

- Points → games → sets
- Match formats
- Tiebreak logic

2. Event Tracker

Tracks:

- Unforced errors
- Double faults
- Winners (optional later)

3. Health Integration

Collects:

- Heart rate
- Steps
- Distance
- Breathing rate (if available)

4. UI Layer

Handles:

- Score display
- Input buttons
- Screens (score / stats / settings)

Phase 2 — Match Engine Design

Match Configuration

Create a config object:

```
MatchConfig:
    setsToWin (2 for best of 3)
    tiebreakEnabled (true/false)
    superTiebreakFinalSet (true/false)
```

Score System

Track:

```
Player:
    points (0,15,30,40)
    games
    sets
    unforcedErrors
    doubleFaults
```

Game Logic

Standard scoring:

- 0 → 15 → 30 → 40 → game
- Deuce handling

Deuce logic:

```
if both == 40:
    state = DEUCE
```

Advantage:

```
if player wins at deuce:
    state = ADVANTAGE_PLAYER
```

🔥 Tiebreak Logic

When:

```
if games == 6-6:  
    start tiebreak
```

Tiebreak rules:

- First to 7 (win by 2)

💥 Super Tiebreak (Final Set)

If enabled:

- Replace final set with:

first to 10 points, win by 2

🎲 Phase 3 — Event Tracking (Your Custom Stats)

This is where your app becomes *valuable*.

Track events manually via buttons:

Events:

- Unforced Error
- Double Fault

```
onUnforcedError(player):  
    player.unforcedErrors += 1  
    opponentWinsPoint()
```

```
onDoubleFault(player):  
    player.doubleFaults += 1  
    opponentWinsPoint()
```

👉 Important:

Both events **automatically award the point to opponent**

Phase 4 — Health Data Integration

Using Connect IQ APIs, you can access:

Available Metrics

From Garmin sensors:

- Heart Rate
- Steps
- Distance
- Calories
- Respiration rate (device-dependent)



Implementation Concept

Start tracking when match starts:

```
Activity.start()  
Sensor.enableHeartRate()
```

During match:

- Poll data periodically
- Or subscribe to updates

```
hr = Sensor.getHeartRate()  
steps = Activity.getSteps()  
distance = Activity.getDistance()
```



Store per match:

```
MatchStats:  
    avgHeartRate  
    maxHeartRate  
    totalSteps  
    totalDistance  
    calories
```

Reality Check

- Not all Vivoactive devices support **breathing rate**
- Sensor polling must be efficient (battery!)

Phase 5 — UI/UX Flow

Garmin screens are small → keep it simple.

Screen Structure

1. Pre-Match Setup

- Choose:
 - Best of 3
 - Tiebreak ON/OFF
 - Super tiebreak ON/OFF

2. Match Screen (Main)

Display:

Player A: 40 | Games: 5 | Sets: 1
Player B: 30 | Games: 4 | Sets: 1

Buttons:

- +Point Player A
- +Point Player B
- UE (A/B)
- DF (A/B)

3. Stats Screen (Swipe)

Unforced Errors:

A: 12 | B: 8

Double Faults:

A: 3 | B: 5

Heart Rate:
Avg: 145 bpm



Phase 6 — Data Persistence

Use storage API:

- Save match state periodically
- Recover if app closes

```
Storage.setValue("matchState", state)
```



Phase 7 — Testing Strategy

Test in layers:

1. Logic tests

- Deuce loops
- Tiebreak transitions
- Super tiebreak trigger

2. UI tests

- Button responsiveness during movement

3. Real-world test

- Play a match and log:
 - Accuracy
 - Ease of use



Phase 8 — Nice-to-Have Enhancements

After MVP:

Smart features:

- Auto-detect serves (advanced)
- Rally length estimation
- Shot classification (very advanced)

Sync:

- Export to phone

- Match history

Suggested Build Order (IMPORTANT)

Don't build everything at once:

- 1  Score system (no stats)
- 2  Add match formats
- 3  Add unforced errors & double faults
- 4  Add health tracking
- 5  Improve UI
- 6  Save/load matches

Complexity Reality

Feature	Difficulty
Score tracking	Easy
Match formats	Medium
Error tracking	Easy
Health integration	Medium
Swing detection	Hard (skip for now)

If you want next step

I can:

- Write a **working starter project structure**
- Give you **real Monkey C code for the scoring engine**
- Or design your **UI layout for Garmin screens**

Just tell me 

Second try

Great — now we've converged on a **clean, realistic, and buildable concept**. Here's your **complete structured build plan** incorporating all your decisions:

-  Simple input: *Won / Error / Double Fault*

-  Undo system
-  Match formats (Bo3, tiebreak, super tiebreak)
-  Timer
-  Health tracking (HR, steps, distance, etc.)

All designed for **Garmin Connect IQ** using **Monkey C**.



0. Final App Concept (What You're Building)

A **tennis match tracker** that:

- Lets you log points in **one tap**
- Tracks full tennis scoring logic
- Records basic match stats + health data
- Is usable during real play (low friction)



1. Architecture Overview

Split your app into **5 modules**:

1. Match Engine

Handles:

- Points → games → sets
- Deuce / advantage
- Tiebreaks & super tiebreaks

2. Input Handler

Processes:

- Won / Error / Double Fault
- Routes to scoring logic

3. History Manager (Undo)

- Stores previous states
- Restores state on undo

4. Health Tracker

- Heart rate
- Steps
- Distance

- Calories

5. UI Layer

- Match screen
- Setup screen
- Stats screen



2. Match Configuration (Pre-Match Screen)



User selects:

- Best of 3 sets
- Tiebreak: ON/OFF
- Super tiebreak final set: ON/OFF

Data Model

```
MatchConfig:
    setsToWin = 2
    tiebreakEnabled = true
    superTiebreakFinalSet = true
```



3. Core Scoring System

Player Structure

```
Player:
    points
    games
    sets

    winners
    unforcedErrors
    doubleFaults
```

Point Input Model (FINAL)

```
enum InputType:
    WON
    ERROR
    DOUBLE_FAULT
```

Input Handling

```
handleInput(type):

    saveStateToHistory()

    if type == WON:
        player.winners += 1
        awardPoint(player)

    elif type == ERROR:
        player.unforcedErrors += 1
        awardPoint(opponent)

    elif type == DOUBLE_FAULT:
        player.doubleFaults += 1
        awardPoint(opponent)
```

Point → Game Logic

Normal scoring:

- 0 → 15 → 30 → 40 → game

Deuce:

```
if both == 40:
    state = DEUCE
```

Advantage:

```
if player.wins at deuce:  
    advantage = player
```

Game → Set Logic

```
if player.games >= 6 and lead >= 2:  
    player.sets += 1
```

Tiebreak

Trigger:

```
if games == 6-6:  
    startTiebreak()
```

Rules:

- First to 7, win by 2

Super Tiebreak

If enabled in final set:

- Replace set with:

first to 10 points, win by 2

4. Undo System (CRITICAL)

Requirement:

- Instant correction during play

Implementation

Use a stack:

```
history = []
```

Save state BEFORE each input:

```
saveState():  
    history.push(copy(currentMatchState))
```

Undo:

```
undo():  
    if history not empty:  
        currentMatchState = history.pop()
```

UX Design

- Long press → Undo last point
or
- Dedicated small “Undo” button

👉 Long press is cleaner for small screens

5. Timer System

Features:

- Match duration
- Always visible

Implementation

```
onMatchStart:  
    startTime = now()
```

```
onUpdate:  
    elapsed = now() - startTime
```

Display

🕒 00:42:18

Optional:

- Pause/resume

❤️ 6. Health Tracking Integration

Using Garmin APIs:

Metrics:

- Heart rate
- Steps
- Distance
- Calories
- (Respiration if available)

Start tracking:

```
Activity.start()  
Sensor.enableHeartRate()
```

During match:

```
hr = Sensor.getHeartRate()  
steps = Activity.getSteps()  
distance = Activity.getDistance()
```

Store summary:

MatchStats:

```
    avgHeartRate
    maxHeartRate
    totalSteps
    totalDistance
    calories
```



7. UI Design (Optimized for Play)



Screen 1 — Setup

- Match format options
- Start match button



Screen 2 — Match (MAIN)

Display:

```
SET: 1-1
GAME: 5-4
POINT: 40-30
```

🕒 00:32:10

Buttons:

```
[ WON ]
[ ERROR ]
[ DF ]
```

Gesture:

- Long press → Undo



Screen 3 — Stats (Swipe)

Winners: 12
Errors: 18
Double Faults: 3

HR Avg: 142 bpm
Steps: 4,230



8. Data Persistence

Save periodically:

```
Storage.setValue("matchState", state)
```

Restore on reopen:

```
state = Storage.getValue("matchState")
```



9. Testing Plan

Phase 1 — Logic

- Deuce loops
- Tiebreak transitions
- Super tiebreak activation

Phase 2 — UX

- Can you log points in < 2 seconds?

Phase 3 — Real Match

- Test during actual play
- Check:
 - Missed taps
 - Need for undo
 - Screen readability



10. Build Order (VERY IMPORTANT)

Follow this exact order:

Step 1



Basic scoring (points/games/sets)

Step 2



Add match formats (tiebreaks)

Step 3



Implement Won/Error/DF input

Step 4



Add Undo system

Step 5



Add Timer

Step 6



Add health tracking

Step 7



Polish UI

Step 8



Save/load match



Final Reality Check

This version is:

-  Fast enough for real matches

-  Simple to implement
-  Expandable later
-  Actually usable on a watch

Next Step

If you want to move forward, I can:

- Build your **scoring engine in real Monkey C**
- Create a **ready-to-run project template**
- Or design your **exact UI layout for Vivoactive (button + touch interactions)**

Just tell me what you want to tackle next 

What I Recommend (Clear Path)

Build a “Playable MVP” in 3 phases

Phase 1 — Core Match Tracker (DO THIS FIRST)

This is your **must-have version**.

Features:

- Won / Error / Double Fault input
- Full tennis scoring (games, sets, tiebreaks)
- Simple timer
- Basic UI (big buttons)
- Undo (last action)

 If you stop here, you already have a **usable tennis app**

Phase 2 — Make It Reliable

Once Phase 1 works in a real match:

Add:

- Match setup (Bo3, tiebreak, super tiebreak)
- Data persistence (resume match)
- Better UI readability (contrast, font size)

 This turns your app from “prototype” → “tool”

Phase 3 — Add Value (Health + Stats)

Only after you trust the app:

Add:

- Heart rate
- Steps & distance
- Post-match stats screen

👉 This is where it becomes *interesting*, not just functional

What NOT to Do (Common Trap)

Avoid starting with:

- Health tracking
- Fancy UI
- Too many stats
- Swing detection

👉 These will slow you down and may never get finished

Why This Order Works

Because your hardest real-world constraint is:

 “Can I log a point in 1–2 seconds while tired?”

If Phase 1 fails that → the rest doesn’t matter.

Practical Milestone

Your first success milestone should be:

“I can play a full set and log every point without frustration.”

Not:

- perfect stats
- beautiful UI
- advanced features

What I Suggest You Do Next (Concrete Step)

👉 Start with:

Step 1:

Build a **minimal scoring engine only**

- No UI yet
- Just functions like:

```
awardPoint(player)
handleInput(WON | ERROR | DF)
```

Test:

- Deuce
- Advantage
- Game win
- Set win

Step 2:

Add a **basic screen with 3 buttons**

- WON
- ERROR
- DF

Print score to screen

Step 3:

Add **Undo**

Only after everything works



If You Want My Help

The most useful next thing I can give you is:

Option A (Best)

👉 A ready-to-use scoring engine in Monkey C

Option B

👉 A project structure + file layout

Option C

👉 A UI mockup tailored to your Garmin screen



My Honest Recommendation

Start with **Option A (scoring engine code)**

Because:

- It's the hardest part to get right
- Everything else depends on it
- It removes 70% of your complexity

👉 **Prioritize a working MVP over features.**

The biggest risk isn't missing features — it's building something too complex to finish or too slow to use during a match.



What I Recommend (Clear Path)



Build a “Playable MVP” in 3 phases



Phase 1 — Core Match Tracker (DO THIS FIRST)

This is your **must-have version**.

Features:

- Won / Error / Double Fault input
- Full tennis scoring (games, sets, tiebreaks)
- Simple timer
- Basic UI (big buttons)
- Undo (last action)

👉 If you stop here, you already have a **usable tennis app**

🟦 Phase 2 — Make It Reliable

Once Phase 1 works in a real match:

Add:

- Match setup (Bo3, tiebreak, super tiebreak)
- Data persistence (resume match)
- Better UI readability (contrast, font size)

👉 This turns your app from “prototype” → “tool”

🟪 Phase 3 — Add Value (Health + Stats)

Only after you trust the app:

Add:

- Heart rate
- Steps & distance
- Post-match stats screen

👉 This is where it becomes *interesting*, not just functional

⚠️ What NOT to Do (Common Trap)

Avoid starting with:

- Health tracking
- Fancy UI
- Too many stats
- Swing detection

👉 These will slow you down and may never get finished

🧠 Why This Order Works

Because your hardest real-world constraint is:

🕒 “Can I log a point in 1–2 seconds while tired?”

If Phase 1 fails that → the rest doesn’t matter.

Practical Milestone

Your first success milestone should be:

“I can play a full set and log every point without frustration.”

Not:

- perfect stats
- beautiful UI
- advanced features

What I Suggest You Do Next (Concrete Step)

👉 Start with:

Step 1:

Build a **minimal scoring engine only**

- No UI yet
- Just functions like:

```
awardPoint(player)
handleInput(WON | ERROR | DF)
```

Test:

- Deuce
- Advantage
- Game win
- Set win

Step 2:

Add a **basic screen with 3 buttons**

- WON
- ERROR
- DF

Print score to screen

Step 3:

Add **Undo**

Only after everything works

If You Want My Help

The most useful next thing I can give you is:

Option A (Best)

👉 A ready-to-use scoring engine in Monkey C

Option B

👉 A project structure + file layout

Option C

👉 A UI mockup tailored to your Garmin screen

My Honest Recommendation

Start with **Option A (scoring engine code)**

Because:

- It's the hardest part to get right
- Everything else depends on it
- It removes 70% of your complexity

Just tell me:

👉 “Give me the scoring engine code”

and I'll build it for you step by step 👍

go ahead, scoring engine code

Perfect — this is the **most important piece** of your app. I'll give you a **clean, working scoring engine** in **Monkey C** style, designed specifically for your input model:

👉 **WON / ERROR / DOUBLE FAULT + Undo + Match formats**

1. Core Design

We'll build a single class:

TennisMatchEngine

Responsibilities:

- Handle scoring (points/games/sets)
- Handle match formats
- Track stats
- Support undo

2. Full Scoring Engine (MVP Code)

```
using Toybox.Lang;

class TennisMatchEngine {

    // ===== ENUMS =====
    const WON = 0;
    const ERROR = 1;
    const DOUBLE_FAULT = 2;

    // ===== MATCH CONFIG =====
    var setsToWin;
    var tiebreakEnabled;
    var superTiebreakFinalSet;

    // ===== STATE =====
    var player;
    var opponent;

    var inTiebreak = false;
    var inSuperTiebreak = false;

    var history; // for undo

    function initialize(config) {
```

```

        setsToWin = config.setsToWin;
        tiebreakEnabled = config.tiebreakEnabled;
        superTiebreakFinalSet =
config.superTiebreakFinalSet;

        player = newPlayer();
        opponent = newPlayer();

        history = [];
    }

function newPlayer() {
    return {
        :points => 0,
        :games => 0,
        :sets => 0,

        :winners => 0,
        :unforcedErrors => 0,
        :doubleFaults => 0
    };
}

// ===== INPUT HANDLER =====
function handleInput(type) {

    saveState();

    if (type == WON) {
        player.winners += 1;
        awardPoint(player, opponent);

    } elseif (type == ERROR) {
        player.unforcedErrors += 1;
        awardPoint(opponent, player);

    } elseif (type == DOUBLE_FAULT) {
        player.doubleFaults += 1;
        awardPoint(opponent, player);
    }
}

// ===== CORE POINT LOGIC =====

```

```
using Toybox.Lang;
```

```
class TennisMatchEngine {
```

```
    // ===== ENUMS =====
```

```
    const WON = 0;
```

```
    const ERROR = 1;
```

```
    const DOUBLE_FAULT = 2;
```

```
    // ===== MATCH CONFIG =====
```

```
    var setsToWin;
```

```
    var tiebreakEnabled;
```

```
    var superTiebreakFinalSet;
```

```
    // ===== STATE =====
```

```
    var player;
```

```
    var opponent;
```

```
    var inTiebreak = false;
```

```
    var inSuperTiebreak = false;
```

```
    var history; // for undo
```

```
    function initialize(config) {
```

```
        setsToWin = config.setsToWin;
```

```
        tiebreakEnabled = config.tiebreakEnabled;
```

```
        superTiebreakFinalSet = config.superTiebreakFinalSet;
```

```

    player = newPlayer();

    opponent = newPlayer();

    history = [];
}

function newPlayer() {

    return {

        :points => 0,

        :games => 0,

        :sets => 0,

        :winners => 0,

        :unforcedErrors => 0,

        :doubleFaults => 0

    };
}

// ===== INPUT HANDLER =====

function handleInput(type) {

    saveState();

    if (type == WON) {

        player.winners += 1;

        awardPoint(player, opponent);

```

```

    } elseif (type == ERROR) {
        player.unforcedErrors += 1;
        awardPoint(opponent, player);

    } elseif (type == DOUBLE_FAULT) {
        player.doubleFaults += 1;
        awardPoint(opponent, player);
    }
}

// ===== CORE POINT LOGIC =====

function awardPoint(winner, loser) {

    if (inSuperTiebreak) {
        winner.points += 1;
        checkSuperTiebreakWin(winner, loser);
        return;
    }

    if (inTiebreak) {
        winner.points += 1;
        checkTiebreakWin(winner, loser);
        return;
    }

    // Normal scoring

```

```
if (winner.points < 3) {  
    winner.points += 1;  
    return;  
}
```

```
// 40-40 (Deuce)
```

```
if (winner.points == 3 && loser.points == 3) {  
    winner.points = 4; // Advantage  
    return;  
}
```

```
// Advantage win
```

```
if (winner.points == 4) {  
    winGame(winner, loser);  
    return;  
}
```

```
// Opponent had advantage → back to deuce
```

```
if (loser.points == 4) {  
    loser.points = 3;  
    return;  
}
```

```
// Win game from 40
```

```
if (winner.points == 3 && loser.points < 3) {  
    winGame(winner, loser);  
}
```

```
}
```

```
// ===== GAME WIN =====
```

```
function winGame(winner, loser) {
```

```
    winner.games += 1;
```

```
    // Reset points
```

```
    winner.points = 0;
```

```
    loser.points = 0;
```

```
    checkSetWin(winner, loser);
```

```
}
```

```
// ===== SET WIN =====
```

```
function checkSetWin(winner, loser) {
```

```
    // Super tiebreak final set
```

```
    if (superTiebreakFinalSet &&
```

```
        winner.sets == setsToWin - 1 &&
```

```
        loser.sets == setsToWin - 1) {
```

```
        inSuperTiebreak = true;
```

```
        winner.points = 0;
```

```
        loser.points = 0;
```

```
        return;
```

```
}
```

```

// Tiebreak

if (tiebreakEnabled && winner.games == 6 && loser.games == 6) {

    inTiebreak = true;

    winner.points = 0;

    loser.points = 0;

    return;

}


// Normal set win

if (winner.games >= 6 && (winner.games - loser.games) >= 2) {

    winner.sets += 1;


    winner.games = 0;

    loser.games = 0;

}

}


// ===== TIEBREAK =====

function checkTiebreakWin(winner, loser) {

    if (winner.points >= 7 && (winner.points - loser.points) >= 2) {

        winner.sets += 1;


        resetAfterSet();

        inTiebreak = false;

    }

}

```

```
// ===== SUPER TIEBREAK =====
```

```
function checkSuperTiebreakWin(winner, loser) {  
  if (winner.points >= 10 && (winner.points - loser.points) >= 2) {  
    winner.sets += 1;  
  
    resetAfterSet();  
    inSuperTiebreak = false;  
  }  
}
```

```
function resetAfterSet() {  
  player.games = 0;  
  opponent.games = 0;  
  
  player.points = 0;  
  opponent.points = 0;  
}
```

```
// ===== UNDO SYSTEM =====
```

```
function saveState() {  
  var snapshot = {  
    :player => clonePlayer(player),  
    :opponent => clonePlayer(opponent),  
    :inTiebreak => inTiebreak,  
    :inSuperTiebreak => inSuperTiebreak  
  };  
}
```

```

    history.add(snapshot);
}

function undo() {
    if (history.size() > 0) {
        var prev = history.remove(history.size() - 1);

        player = prev[:player];
        opponent = prev[:opponent];
        inTiebreak = prev[:inTiebreak];
        inSuperTiebreak = prev[:inSuperTiebreak];
    }
}

```

```

function clonePlayer(p) {
    return {
        :points => p[:points],
        :games => p[:games],
        :sets => p[:sets],
        :winners => p[:winners],
        :unforcedErrors => p[:unforcedErrors],
        :doubleFaults => p[:doubleFaults]
    };
}

```

```

// ===== DISPLAY HELPERS =====

```

```

function getPointDisplay(p) {

```

```

    if (inTiebreak || inSuperTiebreak) {
        return p.points;
    }

    if (p.points == 0) return "0";
    if (p.points == 1) return "15";
    if (p.points == 2) return "30";
    if (p.points == 3) return "40";
    if (p.points == 4) return "Ad";

    return "";
}
}

```



3. How to Use It

Initialize:

```

var config = {
    :setsToWin => 2,
    :tiebreakEnabled => true,
    :superTiebreakFinalSet => true
};

var engine = new TennisMatchEngine(config);

```

On Button Press:

```

engine.handleInput(TennisMatchEngine.WON);

```

```
engine.handleInput(TennisMatchEngine.ERROR);  
engine.handleInput(TennisMatchEngine.DOUBLE_FAULT);
```

Undo:

```
engine.undo();
```

Display Score:

```
var p1 = engine.player;  
var p2 = engine.opponent;  
  
var scoreText =  
    engine.getPointDisplay(p1) + " - " +  
    engine.getPointDisplay(p2);
```

⚠ Known Limitations (Intentional for MVP)

- No match-end detection yet
- No serve tracking
- No forced errors
- No multi-player perspective switching

👉 Keep it simple first — this already handles **real matches**

🚀 What You Should Do Next

Step 1

Plug this into a basic UI:

- 3 buttons: WON / ERROR / DF

Step 2

Render:

- Points
- Games
- Sets

Step 3

Add:

- Long press → `engine.undo()`

I'll give you a **working UI structure** for a Garmin app using **Garmin Connect IQ** with **Monkey C**.



What We're Building Now

A **single main screen** with:

- Score display (sets / games / points)
- Timer
- 3 buttons:
 - WON
 - ERROR
 - DF
- Long press = Undo



1. App Structure (Files)

You'll typically have:

```
App.mc
MainView.mc
TennisMatchEngine.mc    // (your scoring engine)
```



2. Main View (Core UI)

This is where everything comes together.

```
using Toybox.Application;
using Toybox.Graphics;
using Toybox.System;
using Toybox.WatchUi;
```

```

class
using Toybox.Application;

using Toybox.Graphics;

using Toybox.System;

using Toybox.WatchUi;


class MainView extends WatchUi.View {

    var engine;

    var startTime;


    function initialize() {

        View.initialize();


        var config = {

            :setsToWin => 2,

            :tiebreakEnabled => true,

            :superTiebreakFinalSet => true

        };


        engine = new TennisMatchEngine(config);

        startTime = System.getClockTime();

    }


    // ===== DRAW SCREEN =====

    function onUpdate(dc) {

```

```
dc.clear();

var width = dc.getWidth();

var height = dc.getHeight();

// --- TIMER ---

var elapsed = System.getClockTime() - startTime;

var timeStr = formatTime(elapsed);

dc.drawText(width/2, 20, Graphics.FONT_SMALL, timeStr,
Graphics.TEXT_JUSTIFY_CENTER);

// --- SCORE ---

var p1 = engine.player;

var p2 = engine.opponent;

var scoreText =

    engine.getPointDisplay(p1) + " - " +
    engine.getPointDisplay(p2);

var gameText =

    p1.games + " - " + p2.games;

var setText =

    p1.sets + " - " + p2.sets;

dc.drawText(width/2, 60, Graphics.FONT_LARGE, scoreText,
Graphics.TEXT_JUSTIFY_CENTER);
```

```
dc.drawText(width/2, 100, Graphics.FONT_MEDIUM, "Games: " + gameText,  
Graphics.TEXT_JUSTIFY_CENTER);
```

```
dc.drawText(width/2, 130, Graphics.FONT_MEDIUM, "Sets: " + setText,  
Graphics.TEXT_JUSTIFY_CENTER);
```

```
// --- BUTTONS ---
```

```
drawButton(dc, "WON", width/2, height - 120);
```

```
drawButton(dc, "ERROR", width/2, height - 80);
```

```
drawButton(dc, "DF", width/2, height - 40);
```

```
}
```

```
// ===== BUTTON DRAW =====
```

```
function drawButton(dc, label, x, y) {
```

```
    dc.drawText(x, y, Graphics.FONT_MEDIUM, label,  
Graphics.TEXT_JUSTIFY_CENTER);
```

```
}
```

```
// ===== INPUT HANDLING =====
```

```
function onSelect() {
```

```
    // Tap = WON
```

```
    engine.handleInput(TennisMatchEngine.WON);
```

```
    WatchUi.requestUpdate();
```

```
}
```

```
function onSwipe(direction) {
```

```
    if (direction == WatchUi.SWIPE_UP) {
```

```
        engine.handleInput(TennisMatchEngine.WON);
```

```

    } elseif (direction == WatchUi.SWIPE_LEFT) {
        engine.handleInput(TennisMatchEngine.ERROR);
    } elseif (direction == WatchUi.SWIPE_RIGHT) {
        engine.handleInput(TennisMatchEngine.DOUBLE_FAULT);
    }

    WatchUi.requestUpdate();
}

// ===== LONG PRESS = UNDO =====
function onLongPress() {
    engine.undo();
    WatchUi.requestUpdate();
}

// ===== TIME FORMAT =====
function formatTime(ms) {
    var totalSec = ms / 1000;
    var min = totalSec / 60;
    var sec = totalSec mod 60;

    return min + ":" + (sec < 10 ? "0" : "") + sec;
}
}

```



3. Controls (Optimized for Play)

Default Mapping

Action	Input
WON	Tap / Select
ERROR	Swipe Left
DOUBLE FAULT	Swipe Right
UNDO	Long press

Why This Works

- No tiny buttons to hit 🎯
- Works while moving
- Minimal thinking
- Fast recovery (undo)

You May Want to Swap This

Some players prefer:

Action	Alternative
WON	Swipe Up
ERROR	Swipe Down
DF	Tap

👉 You can easily remap later

4. Timer — Already Integrated

You already have:

```
startTime = System.getClockTime();
```

And:

```
elapsed = System.getClockTime() - startTime;
```

5. Next Step: Add Health Tracking

We'll integrate:

- Heart rate
- Steps
- Distance

👉 But ONLY after this UI feels good

6. What You Should Test Now

Simulator:

- Tap → score updates
- Swipe → correct event
- Long press → undo works

Real watch:

- Can you log a point in <2 seconds?
- Can you undo instantly?
- Is text readable mid-match?

7. Immediate Improvements (Next Iteration)

Once this works:

Add:

- Bigger button zones (not just text)
- Color feedback (green/red flash)
- Vibration on input

👉 Next Step (I recommend this)

Now that you have:

-  Scoring engine
-  UI
-  Controls

👉 Next, I can help you add:

🔥 Option A

Health tracking (heart rate, steps)

🔥 Option B

Match setup screen (Bo3, tiebreak toggle)

🕒 Priority Roadmap (Strict Order)

We'll follow this order:

✅ 1. Make input rock-solid (UI + usability)

✅ 2. Add match setup (formats)

➡️ SOON 3. Add persistence (resume match)

➡️ SOON 4. Add health tracking

➡️ SOON 5. Polish + stats screen

🎯 Step 1 (NOW): Fix UI for Real Match Use

Your current UI works, but it's **not yet match-proof**.

👉 Problem:

- Text buttons are too small
- No feedback when tapping
- Easy to mis-tap

🟢 Goal: “Blind Usable UI”

You should be able to:

- Tap without looking too much

- Feel confirmation
- Recover instantly

Improved UI Design

We'll switch from text-only → **touch zones**

Screen Layout

SCORE		
40	-	30
G: 5-4	S: 1-1	
WON		
ERROR		DF

← big zone

Updated UI Code (Better Touch Areas + Feedback)

Replace your `onUpdate` and input logic with this improved version:

```
using Toybox.Graphics;
using Toybox.WatchUi;
using Toybox.System;

class MainView extends WatchUi.View {

    var engine;
    using Toybox.Graphics;

    using Toybox.WatchUi;

    using Toybox.System;

    class MainView extends WatchUi.View {
```

```

var engine;

var startTime;

var lastAction = "";

function initialize() {
    View.initialize();

    var config = {
        :setsToWin => 2,
        :tiebreakEnabled => true,
        :superTiebreakFinalSet => true
    };

    engine = new TennisMatchEngine(config);
    startTime = System.getClockTime();
}

function onUpdate(dc) {

    dc.clear();

    var w = dc.getWidth();
    var h = dc.getHeight();

    // ===== TIMER =====

    var elapsed = System.getClockTime() - startTime;

```

```

var timeStr = formatTime(elapsed);

dc.drawText(w/2, 10, Graphics.FONT_SMALL, timeStr,
Graphics.TEXT_JUSTIFY_CENTER);

// ===== SCORE =====

var p1 = engine.player;
var p2 = engine.opponent;

var score =

    engine.getPointDisplay(p1) + " - " +
    engine.getPointDisplay(p2);

var games = p1.games + " - " + p2.games;
var sets = p1.sets + " - " + p2.sets;

dc.drawText(w/2, 40, Graphics.FONT_LARGE, score,
Graphics.TEXT_JUSTIFY_CENTER);

dc.drawText(w/2, 80, Graphics.FONT_MEDIUM, "G: " + games,
Graphics.TEXT_JUSTIFY_CENTER);

dc.drawText(w/2, 110, Graphics.FONT_MEDIUM, "S: " + sets,
Graphics.TEXT_JUSTIFY_CENTER);

// ===== BUTTON ZONES =====

// WON (top big area)

dc.fillRect(0, h/2, w, h/4);

dc.drawText(w/2, h/2 + 20, Graphics.FONT_MEDIUM, "WON",
Graphics.TEXT_JUSTIFY_CENTER);

```

```

// ERROR (bottom left)

dc.fillRectangle(0, h*3/4, w/2, h/4);

dc.drawText(w/4, h*3/4 + 20, Graphics.FONT_MEDIUM, "ERROR",
Graphics.TEXT_JUSTIFY_CENTER);


// DF (bottom right)

dc.fillRectangle(w/2, h*3/4, w/2, h/4);

dc.drawText(w*3/4, h*3/4 + 20, Graphics.FONT_MEDIUM, "DF",
Graphics.TEXT_JUSTIFY_CENTER);


// ===== LAST ACTION FEEDBACK =====

dc.drawText(w/2, h - 10, Graphics.FONT_SMALL, lastAction,
Graphics.TEXT_JUSTIFY_CENTER);
}


// ===== TOUCH INPUT =====

function onTap(clickEvent) {

    var x = clickEvent.getX();

    var y = clickEvent.getY();


    var w = self.getWidth();

    var h = self.getHeight();


    saveAndHandle(x, y, w, h);

    WatchUi.requestUpdate();

}

```

```

function saveAndHandle(x, y, w, h) {

    if (y > h/2 && y < h*3/4) {

        engine.handleInput(TennisMatchEngine.WON);

        lastAction = "WON";

    } elseif (y >= h*3/4 && x < w/2) {

        engine.handleInput(TennisMatchEngine.ERROR);

        lastAction = "ERROR";

    } elseif (y >= h*3/4 && x >= w/2) {

        engine.handleInput(TennisMatchEngine.DOUBLE_FAULT);

        lastAction = "DF";

    }

    System.vibrate();

}

// ===== UNDO =====

function onLongPress() {

    engine.undo();

    lastAction = "UNDO";

    System.vibrate();

    WatchUi.requestUpdate();

}

```

```
function formatTime(ms) {  
    var total = ms / 1000;  
    var min = total / 60;  
    var sec = total mod 60;  
  
    return min + ":" + (sec < 10 ? "0" : "") + sec;  
}  
}
```

What You Just Improved

Before:

- Small targets
- Easy to miss
- No feedback

Now:

-  Big touch zones
-  Visual feedback (“WON”, etc.)
-  Vibration confirmation
-  Undo always available

Your Task Now (IMPORTANT)

Test this version and ask:

1.

Can I log a point **without thinking**?

2.

Can I recover instantly with undo?

3.

Can I use it while slightly out of breath?

Next Step (Step 2)

Once you're happy with input:

👉 We move to:

Match Setup Screen

- Choose:
 - Best of 3
 - Tiebreak ON/OFF
 - Super tiebreak ON/OFF

The goal here is to let the player **choose match format before starting**, then move seamlessly into the main match screen.

1. Match Setup Requirements

Player must select:

- 1 **Sets to Win** – usually Best of 3 (first to 2 sets)
- 2 **Tiebreak** – ON / OFF
- 3 **Super Tiebreak for Final Set** – ON / OFF

Optional: later we could add serve-side preference, but keep MVP simple.

2. Match Setup UI Layout

MATCH SETUP	
Sets to Win: 2	<- Tap to toggle (1 / 2 / 3)
Tiebreak: ON	<- Tap to toggle
Super TB Final: ON	<- Tap to toggle
[START MATCH]	<- Tap to proceed



3. Setup Screen Code (Monkey C)

```
using Toybox.WatchUi;
using Toybox.Graphics;
using Toybox.System;

class SetupView extends WatchUi.View {

    var setsToWin = 2;
    var tiebreak = true;
    var superTB = true;

    function initialize() {
        View.initialize();
    }

    // ===== DRAW SCREEN =====
    function onUpdate(dc) {
        dc.clear();
        var w = dc.getWidth();
        var y = 20;

        dc.drawText(w/2, y, Graphics.FONT_MEDIUM, "MATCH
SETUP", Graphics.TEXT_JUSTIFY_CENTER);
        y += 30;

        dc.drawText(w/2, y, Graphics.FONT_SMALL, "Sets to
Win: " + setsToWin, Graphics.TEXT_JUSTIFY_CENTER);
        y += 25;

        dc.drawText(w/2, y, Graphics.FONT_SMALL,
"Tiebreak: " + (tiebreak ? "ON" : "OFF"),
Graphics.TEXT_JUSTIFY_CENTER);
        y += 25;

        dc.drawText(w/2, y, Graphics.FONT_SMALL, "Super
TB Final: " + (superTB ? "ON" : "OFF"),
Graphics.TEXT_JUSTIFY_CENTER);
        y += 40;
```

```
        dc.drawText(w/2, y, Graphics.FONT_MEDIUM,
"[ START MATCH ]", Graphics.TEXT_JUSTIFY_CENTER);
    }
```

How This Works

- 1 Tap each line to toggle:
 - Sets cycles $1 \rightarrow 2 \rightarrow 3$
 - Tiebreak and Super TB toggle ON/OFF
- 2 Tap **START MATCH** $\rightarrow$ opens **MainView** with your config
- 3 Your **MainView** engine will now respect these settings.

Testing Checklist

- Can I toggle sets?
- Can I toggle tiebreak?
- Can I toggle super tiebreak?
- **START MATCH** opens **MainView** with correct settings

Next Step (Step 3)

Once the **setup screen works**, we move to:

Data persistence

- Save ongoing match (score, points, time)
- Resume after closing app or watch reboot

This ensures you **never lose a match mid-play**, which is critical for real tennis tracking.

1. Garmin Connect IQ & Activities

Garmin watches have a **Toybox.Activities API**:

- **Activity types**: TENNIS is available on most watches.
- Your app can **start, pause, resume, stop an activity**.
- While activity is running, you can **push custom metrics** (like points, winners, errors, heart rate).

Benefits:

- Automatically appears in **Garmin Connect**
- Can be exported via Connect (CSV/FIT)
- Can include **health metrics** like HR, steps, distance



2. Workflow for Your Tennis Tracker

1 Start activity when match starts

```
using Toybox.Activities;  
  
2  
3 var activity = new Activities.Activity();  
4 activity.start(Activities.SPORT_TENNIS);
```

5 During match

- Track points and stats in your engine
- Push updates to activity fields:

```
activity.setMetric("winners",  
engine.player.winners);  
  
◦ activity.setMetric("errors",  
engine.player.unforcedErrors);  
◦ activity.setMetric("double_faults",  
engine.player.doubleFaults);
```

6 Stop activity when match ends

```
activity.stop();
```

7 Sync to Garmin Connect

- Automatically happens when watch syncs
- You can see stats, export FIT/CSV



Notes & Tips

- **Timer integration:** You already have it in `MainView` — push elapsed time as `duration` metric.
- **Health integration:** Use `Toybox.Sensors` API for heart rate, steps, distance during activity.
- **Export:** Once logged, Garmin Connect allows export:
 - **FIT** — native Garmin format
 - **CSV** — easy for Excel
 - **TCX** — compatible with training apps
- **Custom fields:** You can create custom metrics for:
 - Points won
 - Errors

- Double faults
- Total games/sets



Implementation Example

```
using Toybox.Activities;
using Toybox.Sensors;
using Toybox.Activities;

using Toybox.Sensors;

class TennisTracker {

    var activity;

    function start() {
        activity = new Activities.Activity();
        activity.start(Activities.SPORT_TENNIS);
    }

    function updateStats(engine) {
        activity.setMetric("winners", engine.player.winners);
        activity.setMetric("errors", engine.player.unforcedErrors);
        activity.setMetric("double_faults", engine.player.doubleFaults);

        // Optional health metrics
        var hr = Sensors.HeartRate.getCurrent();
        var steps = Sensors.StepCounter.getTotalSteps();
        activity.setMetric("heart_rate", hr);
        activity.setMetric("steps", steps);
    }
}
```

```

    }

    function stop() {
        activity.stop();
    }
}

```



Key Takeaways

- 1 Your tennis match can **be a logged Garmin activity**
- 2 Health metrics and tennis stats can **live in the same activity**
- 3 Garmin Connect allows **export to FIT/CSV/TCX**
- 4 This makes your app **fully trackable, shareable, and analyzable**



1. New Architecture

We'll add a **TennisActivityManager** that handles:

- Start / stop of activity
- Live updates of:
 - Winners
 - Unforced Errors
 - Double Faults
 - Heart Rate
 - Steps / Distance
- Auto-push updates on each point

It will be called by your **MainView + TennisMatchEngine**.



2. Activity Manager Code

```

using Toybox.Activities;
using Toybox.Activities;

using Toybox.Sensors;

using Toybox.System;

```

```
class TennisActivityManager {

    var activity;

    function initialize() {
        activity = new Activities.Activity();
    }

    // Start tennis activity
    function start() {
        activity.start(Activities.SPORT_TENNIS);
        System.println("Tennis activity started");
    }

    // Push match stats and optional health metrics
    function update(engine) {

        // Tennis stats
        activity.setMetric("winners", engine.player.winners);
        activity.setMetric("errors", engine.player.unforcedErrors);
        activity.setMetric("double_faults", engine.player.doubleFaults);

        // Optional health metrics
        if (Sensors.HeartRate.isAvailable()) {
            var hr = Sensors.HeartRate.getCurrent();
            activity.setMetric("heart_rate", hr);
        }
    }
}
```

```

        if (Sensors.StepCounter.isAvailable()) {
            var steps = Sensors.StepCounter.getTotalSteps();
            activity.setMetric("steps", Sensors.StepCounter.getTotalSteps());
        }

        if (Sensors.Gps.isAvailable()) {
            var dist = Sensors.Gps.getDistance();
            activity.setMetric("distance", dist);
        }
    }

    // Stop activity
    function stop() {
        activity.stop();
        System.println("Tennis activity stopped");
    }
}

```

3. Integrate into MainView

Update **MainView**:

```

// In MainView.initialize()
activityManager = new TennisActivityManager();
activityManager.start();

// In handleInput() (after engine.handleInput)
activityManager.update(engine);

// In match end or on stop button
activityManager.stop();

```



4. Workflow During Match

- 1 **Start Match:** `TennisActivityManager.start()` → Garmin activity begins
- 2 **Point Input:** WON / ERROR / DF → engine updates → `activityManager.update()`
- 3 **Health & Steps:** Recorded live in the same activity
- 4 **End Match:** `activityManager.stop()` → all data saved to Garmin Connect
- 5 **Export:** Users can export FIT / CSV / TCX



5. Advantages

- Every point is logged live
- Health metrics integrated
- Full activity in Garmin Connect, exportable
- Undo still works — updates activity each time



Next Step (Optional Enhancements)

Once this is working:

- 1 **Auto-pause/resume** when the watch detects inactivity
- 2 **Vibration feedback** when a point is logged
- 3 **Post-match summary screen:**
 - Total points won
 - Errors & double faults
 - Heart rate stats
 - Duration

Perfect — let's build the **Post-Match Summary Screen**. This will display all the key stats from the match **immediately after the activity ends**, using data from your `TennisMatchEngine` and `TennisActivityManager`.

This screen will also serve as a **visual summary** before syncing/exporting to Garmin Connect.



1. What We'll Display

Match Stats:

Metric	Description
Sets	Player vs Opponent
Games	Player vs Opponent
Points	Player vs Opponent

Winners	Total winners
Unforced Errors	Total unforced errors
Double Faults	Total double faults
Duration	Match timer
Heart Rate	Avg & Max (if sensor available)
Steps	Total steps (if sensor available)

Optional: later add **distance** or calories.

2. Summary Screen Layout

```

-----
|           MATCH SUMMARY           |
| Sets: 2 - 1                       |
| Games: 6 - 4                     |
| Points: 40 - 15                   |
| Winners: 12                      |
| Unforced Errors: 3                |
| Double Faults: 1                 |
| Duration: 45:30                  |
| Heart Rate: 140/160              |
| Steps: 1200                      |
|                                   |
| [ DONE / EXIT ]                  |
|                                   |
-----

```

3. PostMatchView Code (Monkey C)

```

using Toybox.WatchUi;
usi
using Toybox.WatchUi;

using Toybox.Graphics;

using Toybox.System;

class PostMatchView extends WatchUi.View {

```

```
var engine;

var activityManager;

var startTime;

var endTime;


var avgHR = 0;

var maxHR = 0;

var steps = 0;


function initialize(engineObj, activityMgr, start) {

    View.initialize();


    engine = engineObj;

    activityManager = activityMgr;

    startTime = start;

    endTime = System.getClockTime();


    // Capture health metrics if available
    if (Sensors.HeartRate.isAvailable()) {

        avgHR = Sensors.HeartRate.getAverage();

        maxHR = Sensors.HeartRate.getMax();

    }

    if (Sensors.StepCounter.isAvailable()) {

        steps = Sensors.StepCounter.getTotalSteps();

    }

}
```

```

// ===== DRAW SCREEN =====

function onUpdate(dc) {

    dc.clear();

    var w = dc.getWidth();

    var y = 20;


    dc.drawText(w/2, y, Graphics.FONT_MEDIUM, "MATCH SUMMARY",
Graphics.TEXT_JUSTIFY_CENTER);

    y += 30;


    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Sets: " + engine.player.sets + " - " +
engine.opponent.sets, Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;


    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Games: " + engine.player.games + " - "
+ engine.opponent.games, Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;


    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Points: " +
engine.getPointDisplay(engine.player) + " - " + engine.getPointDisplay(engine.opponent),
Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;


    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Winners: " + engine.player.winners,
Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;


    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Unforced Errors: " +
engine.player.unforcedErrors, Graphics.TEXT_JUSTIFY_CENTER);

```

```

    y += 20;

    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Double Faults: " +
engine.player.doubleFaults, Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;

    var duration = formatTime(endTime - startTime);

    dc.drawText(w/2, y, Graphics.FONT_SMALL, "Duration: " + duration,
Graphics.TEXT_JUSTIFY_CENTER);

    y += 20;

    if (avgHR > 0) {

        dc.drawText(w/2, y, Graphics.FONT_SMALL, "Heart Rate: " + avgHR + "/" +
maxHR, Graphics.TEXT_JUSTIFY_CENTER);

        y += 20;

    }

    if (steps > 0) {

        dc.drawText(w/2, y, Graphics.FONT_SMALL, "Steps: " + steps,
Graphics.TEXT_JUSTIFY_CENTER);

    }

    // DONE button at bottom

    dc.drawText(w/2, dc.getHeight() - 20, Graphics.FONT_SMALL, "[ DONE ]",
Graphics.TEXT_JUSTIFY_CENTER);

}

function onTap(clickEvent) {

    // Exit summary

```

```

    WatchUi.popView();
}

function formatTime(ms) {
    var totalSec = ms / 1000;
    var min = totalSec / 60;
    var sec = totalSec mod 60;
    return min + ":" + (sec < 10 ? "0" : "") + sec;
}
}

```

4. How It Integrates

- After **match ends**:

```

// Stop activity first
activityManager.stop();

// Push PostMatchView
var postView = new PostMatchView(engine, activityManager,
    startTime);
WatchUi.pushView(postView);

```

- All stats are **live and final**, pulled from `TennisMatchEngine` and activity manager

5. Benefits

- 1 **Instant summary** right after match
- 2 Stats match **Garmin Connect** activity
- 3 Users can **review winners, errors, DF, duration, heart rate, steps**
- 4 Optional: **export via Garmin Connect** for sharing or analysis

Perfect — now we'll implement **match persistence and resume**, so your tennis app can handle:

- Watch reboot or app close mid-match
- Pausing/resuming a match
- Ensuring Garmin activity continues recording

This is **Step 4** in our roadmap.

1. Persistence Strategy

Garmin Connect IQ provides **Toybox.Storage**, which lets you save small datasets **locally** on the watch.

We'll save:

- **TennisMatchEngine** state (points, games, sets, winners, errors, double faults)
- **startTime** / elapsed match time
- Health metrics (optional)
- Match configuration (sets to win, tiebreak, super tiebreak)

2. Persistence Manager Code

```
using Toybox.Storage;
```

```
class MatchPersistence {
```

```
    const STORAGE_KEY = "TennisMatch";
```

```
    // Save match state
```

```
    static function save(engine, config, startTime) {
```

```
        var data = {
```

```
            :engineState => engine.getState(), // custom function in TennisMatchEngine
```

```
            :config => config,
```

```
            :startTime => startTime
```

```
        };
```

```
        Storage.setValue(STORAGE_KEY, data);
```

```
        System.println("Match saved");
```

```

    }

    // Load match state
    static function load() {
        if (Storage.hasValue(STORAGE_KEY)) {
            var data = Storage.getValue(STORAGE_KEY);
            return data;
        }
        return null;
    }

    // Clear saved match
    static function clear() {
        Storage.removeValue(STORAGE_KEY);
    }
}

```

3. Integrate with TennisMatchEngine

Engine must implement:

```

class TennisMatchEngine {

    // Return serializable state
    function getState() {
        return {
            :player => player.serialize(),
            :opponent => opponent.serialize(),
            :currentSet => currentSet,

```

```

        :currentGame => currentGame,
        :currentPoint => currentPoint
    };
}

// Restore from saved state
function restore(state) {
    player.deserialize(state.player);
    opponent.deserialize(state.opponent);
    currentSet = state.currentSet;
    currentGame = state.currentGame;
    currentPoint = state.currentPoint;
}
}

```

- `serialize()` → convert object to basic types (integers, booleans)
- `deserialize()` → restore object state

4. Using Persistence in MainView

```

// On initialize
var saved = MatchPersistence.load();
if (saved != null) {
    engine.restore(saved.engineState);
    config = saved.config;
    startTime = saved.startTime;
} else {
    startTime = System.getClockTime();
}

```

```
// After each point / score update

activityManager.update(engine);

MatchPersistence.save(engine, config, startTime);
```

```
// On match end
```

```
MatchPersistence.clear();
```



5. Workflow

- 1 **User opens app**
 - If saved match exists → prompt: **Resume match?**
 - Else → start new match
- 2 **During match**
 - Every point logged → **engine state saved automatically**
 - Garmin activity continues recording
- 3 **If watch closes / app quits**
 - Resume from **last saved state**
- 4 **At match end**
 - Stop activity → show **Post-Match Summary**
 - Clear stored state



6. Benefits

- No points lost if watch reboots
- Match continues seamlessly after pause
- Garmin activity continues in the background
- Full integration with **Post-Match Summary**



Adjusted Approach: Color-Coded Only

- 1 Keep the **color flash bar** at the top (green/red/orange/yellow)
- 2 Optional: flash the **entire top of the screen** briefly to catch the eye
- 3 Remove all `System.vibrate()` calls in `saveAndHandle()`



Updated `saveAndHandle` Without Vibration

```
function saveAndHandle(x, y, w, h) {
```

```

if (y > h/2 && y < h*3/4) {

    engine.handleInput(TennisMatchEngine.WON);

    lastAction = "WON";

    lastActionColor = Graphics.COLOR_GREEN;

} elseif (y >= h*3/4 && x < w/2) {

    engine.handleInput(TennisMatchEngine.ERROR);

    lastAction = "ERROR";

    lastActionColor = Graphics.COLOR_RED;

} elseif (y >= h*3/4 && x >= w/2) {

    engine.handleInput(TennisMatchEngine.DOUBLE_FAULT);

    lastAction = "DF";

    lastActionColor = Graphics.COLOR_ORANGE;

}

}

function saveAndHandle(x, y, w, h) {

    if (y > h/2 && y < h*3/4) {

        engine.handleInput(TennisMatchEngine.WON);

        lastAction = "WON";

        lastActionColor = Graphics.COLOR_GREEN;

    } elseif (y >= h*3/4 && x < w/2) {

        engine.handleInput(TennisMatchEngine.ERROR);

        lastAction = "ERROR";

        lastActionColor = Graphics.COLOR_RED;

    }

```

```
} elseif (y >= h*3/4 && x >= w/2) {  
    engine.handleInput(TennisMatchEngine.DOUBLE_FAULT);  
    lastAction = "DF";  
    lastActionColor = Graphics.COLOR_ORANGE;  
}  
}
```

